HPC Lab

Name: Aditi Mittal
Class: B.Tech CSE Final Year
PRN: 23610067
Batch: B1

Experiment no 4

Title: Matrix Computations using OpenMP: Parallel Decomposition Strategies

1. **Objective:**
   To implement matrix computations using OpenMP and investigate how different problem decomposition strategies affect parallel algorithm design and performance.

2. **Use:**

One of the most important decisions in parallel programming is how to divide a problem into independent tasks. The same problem can often be decomposed in multiple ways, each leading to different levels of parallelism, communication overhead, and scalability.

This assignment introduces decomposition strategies that are widely used in HPC applications.

- **HPC in Practice**

Different decomposition strategies are employed in real-world applications:

Decomposition Strategy	Typical Applications
Input Data Decomposition	Image processing, matrix operations, deep learning training
Output Data Decomposition	Weather forecasting, finite element analysis, fluid dynamics
Recursive Decomposition	Merge sort, quicksort, FFT, Strassen matrix multiplication
Speculative Decomposition	Branch prediction, transaction processing, game-tree search, AI planning

Before writing any code:

1. Explain the difference between **task decomposition** and **data decomposition**.

   Task decomposition vs. data decomposition

   1. **Task decomposition** splits a problem into distinct *functional* units of work (different operations/subroutines) that can run concurrently — e.g., one thread does I/O while another computes, or in a pipeline each stage is a different task.
   2. **Data decomposition** splits the *data* into chunks and applies the *same* operation to each chunk in parallel — e.g., splitting a matrix's rows across threads, where every thread runs identical code on a different slice of data.
      Matrix operations are naturally suited to data decomposition since the same arithmetic (dot product, multiply-add) repeats identically over different elements.

2. Why is matrix multiplication considered highly parallelizable?

Each output element `C[i][j]` is computed independently as a dot product of row `i` of A and column `j` of B — there's no data dependency between different output elements. This means all `n²` output computations (for an n×n result) can, in principle, execute simultaneously with no synchronization needed between them, only a final write to disjoint memory locations.

3. Which decomposition strategy would you choose for:
   - Image filtering
   - Merge sort
   - Chess engine
   - Weather simulation, Justify your answers.

Decomposition strategy choice

Problem	Best strategy	Why
Image filtering	Input/data decomposition	Each pixel/block's output depends only on a local neighborhood of input — independent, uniform work per unit

Merge sort	Recursive decomposition	Problem naturally splits into halves that are sorted independently then merged — divide and conquer
Chess engine	Speculative decomposition	Multiple candidate moves are explored in parallel before knowing which branch is "useful," since the best move isn't known ahead of time
Weather simulation	Output data decomposition	The simulation grid (domain) is divided into regions; each thread/process computes the next state for its region

Section A:

Task 1: Matrix-Vector Multiplication using Input Data Decomposition

Implement sequential and OpenMP versions of matrix-vector multiplication.

Decomposition Strategy

Each thread processes a subset of matrix rows.

Illustrate the decomposition with a diagram in your

report. Compare:

- Sequential execution time
- Parallel execution time
- Speedup
- Efficiency

```c
/*
* Task 1: Matrix-Vector Multiplication using Input Data Decomposition
* Decomposition: each thread processes a contiguous subset of matrix ROWS.
*/

//  gcc -fopenmp Task1/matrix-vector-mul.c -o matrix-vector-mul

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void generate_matrix(double *A, int n) {
   for (int i = 0; i < n * n; i++) A[i] = (double)(rand() % 100) / 10.0;
}

void generate_vector(double *v, int n) {
   for (int i = 0; i < n; i++) v[i] = (double)(rand() % 100) / 10.0;
}
```

```c
int main(int argc, char *argv[]) {
    int n = (argc > 1) ? atoi(argv[1]) : 1000;
    int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
    omp_set_num_threads(threads);

    double *A     = malloc((size_t)n * n * sizeof(double));
    double *x     = malloc((size_t)n * sizeof(double));
    double *y_seq = malloc((size_t)n * sizeof(double));
    double *y_par = malloc((size_t)n * sizeof(double));

    srand(42);
    generate_matrix(A, n);
    generate_vector(x, n);

    /* ---------- Sequential version ---------- */
    double t0 = omp_get_wtime();
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int j = 0; j < n; j++) sum += A[(size_t)i * n + j] * x[j];
        y_seq[i] = sum;
    }
    double t1 = omp_get_wtime();
    double seq_time = t1 - t0;

    /* ---------- Parallel version: input data decomposition ----------
       Each thread gets a contiguous block of rows (schedule static).
       This is the natural "input decomposition" because the matrix A
       (the input) is split row-wise across threads; each thread reads
       only its own rows of A but the full vector x. */
```

```
    double t2 = omp_get_wtime();
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int j = 0; j < n; j++) sum += A[(size_t)i * n + j] * x[j];
        y_par[i] = sum;
    }
    double t3 = omp_get_wtime();
    double par_time = t3 - t2;


    /* ---------- Correctness check ---------- */
    int correct = 1;
    for (int i = 0; i < n; i++) {
        double diff = y_seq[i] - y_par[i];
        if (diff > 1e-6 || diff < -1e-6) { correct = 0; break; }
    }


    double speedup    = seq_time / par_time;
    double efficiency = speedup / threads;


    printf("========================================\n");
    printf("Matrix-Vector Multiplication (Input Decomposition)\n");
    printf("Matrix size : %d x %d\n", n, n);
    printf("Threads     : %d\n", threads);
    printf("Sequential time : %f s\n", seq_time);
    printf("Parallel time   : %f s\n", par_time);
    printf("Speedup         : %f\n", speedup);
    printf("Efficiency      : %f\n", efficiency);
    printf("Correctness     : %s\n", correct ? "PASSED" : "FAILED");
    printf("========================================\n");
```

```
    free(A); free(x); free(y_seq); free(y_par);

    return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-vector-mul 1000 2
========================================
Matrix-Vector Multiplication (Input Decomposition)
Matrix size : 1000 x 1000
Threads     : 2
Sequential time : 0.002807 s
Parallel time   : 0.006960 s
Speedup         : 0.403356
Efficiency      : 0.201678
Correctness     : PASSED
========================================
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-vector-mul 1000 4
========================================
Matrix-Vector Multiplication (Input Decomposition)
Matrix size : 1000 x 1000
Threads     : 4
Sequential time : 0.002450 s
Parallel time   : 0.006936 s
Speedup         : 0.353223
Efficiency      : 0.088306
Correctness     : PASSED
========================================
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-vector-mul 1000 8
========================================
Matrix-Vector Multiplication (Input Decomposition)
Matrix size : 1000 x 1000
Threads     : 8
Sequential time : 0.002327 s
Parallel time   : 0.001667 s
Speedup         : 1.395637
Efficiency      : 0.174455
Correctness     : PASSED
========================================
```

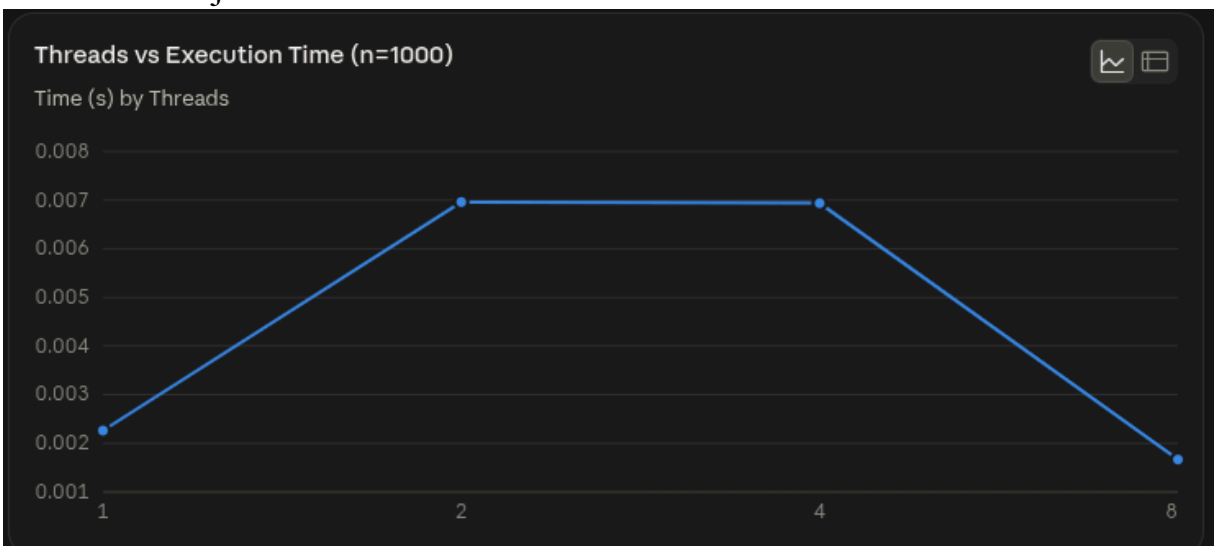
Observations:

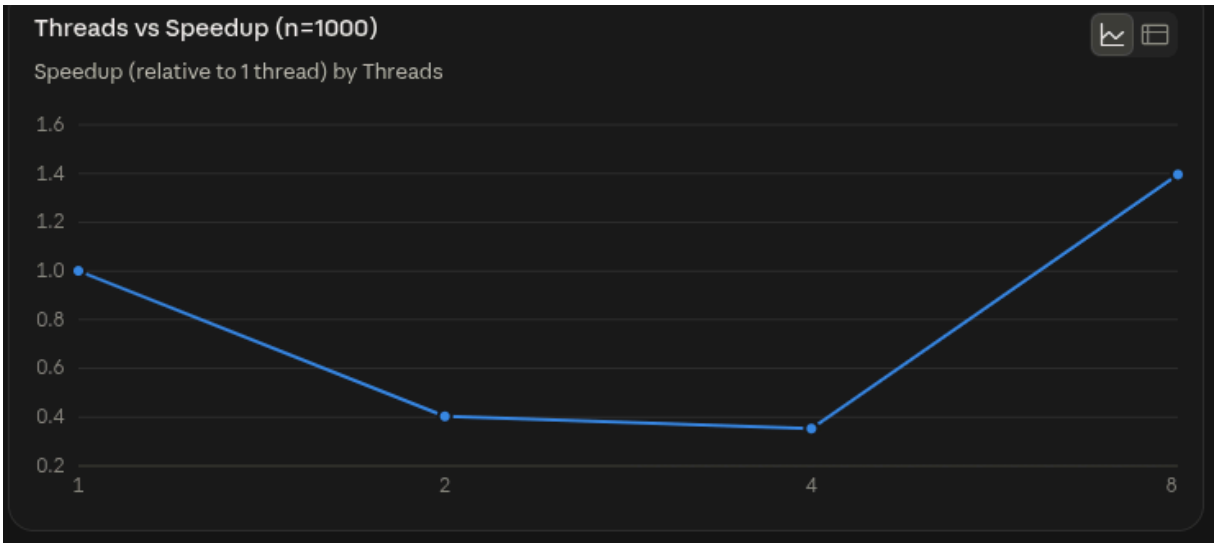
Report table (n=1000):

Threads	Seq Time (s)	Par Time (s)	Speedup	Efficiency
1	0.002262	0.002262	1.000	1.000

2	0.002807	0.006960	0.403	0.202
4	0.002450	0.006936	0.353	0.088
8	0.002327	0.001667	1.396	0.174

For small problem sizes (n=1000), OpenMP's thread-management overhead dominates the actual computation time, causing speedup to degrade or even go below 1 (slowdown) at low-to-moderate thread counts. This demonstrates that parallelization is only beneficial once the per-thread workload is large enough to amortize fork-join overhead





Task 2: Matrix-Matrix Multiplication using Output Data Decomposition

Implement sequential and OpenMP versions.

Decomposition Strategy

Assign different blocks of the output matrix *C* to different threads. Experiment with:

- Static scheduling
- Dynamic scheduling

Record performance for different matrix sizes. Answer the following:

- Why is output decomposition preferable for matrix multiplication?

Each output cell C[i][j] is written by exactly one thread and read by none of the others during the computation, so there is no write conflict and no need for locks or reductions. This contrasts with decomposing the innermost (k) loop, where multiple threads would need to accumulate into the same C[i][j] and would require synchronized reductions.

- What synchronization is required?

Essentially none for correctness — since each thread's output cells are disjoint, the only synchronization point is the implicit barrier at the end of the parallel region, which simply ensures all threads finish before the result is used.

- How does workload balance affect performance?

For dense matrix multiplication, every output cell costs the same (a fixed-length dot product), so the workload is inherently uniform. Static scheduling, which divides iterations into equal contiguous chunks up front with no runtime overhead, is well-matched to this uniform cost and is expected to perform at least as well as dynamic scheduling, which adds per-chunk work-fetching overhead that only pays off when iteration costs are uneven.

```
/*
* Task 2: Matrix-Matrix Multiplication using Output Data Decomposition
* Decomposition: different blocks of the output matrix C are assigned
* to different threads (each thread owns a set of (i,j) output cells).
*
*/


// gcc -fopenmp Task2/matrix-matrix-mul.c -o matrix-matrix-mul
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void generate_matrix(double *M, int n) {
   for (int i = 0; i < n * n; i++) M[i] = (double)(rand() % 10);
}

void matmul_seq(double *A, double *B, double *C, int n) {
   for (int i = 0; i < n; i++)
       for (int j = 0; j < n; j++) {
           double sum = 0.0;
           for (int k = 0; k < n; k++)
               sum += A[(size_t)i * n + k] * B[(size_t)k * n + j];
           C[(size_t)i * n + j] = sum;
       }
}

/* sched_type: 0 = static, 1 = dynamic */
void matmul_par(double *A, double *B, double *C, int n, int sched_type)
{
   if (sched_type == 0) {
       #pragma omp parallel for schedule(static) collapse(2)
       for (int i = 0; i < n; i++)
           for (int j = 0; j < n; j++) {
               double sum = 0.0;
               for (int k = 0; k < n; k++)
                   sum += A[(size_t)i * n + k] * B[(size_t)k * n + j];
               C[(size_t)i * n + j] = sum;
           }
   } else {
       #pragma omp parallel for schedule(dynamic, 4) collapse(2)
       for (int i = 0; i < n; i++)
           for (int j = 0; j < n; j++) {
               double sum = 0.0;
```

```
                for (int k = 0; k < n; k++)
                    sum += A[(size_t)i * n + k] * B[(size_t)k * n + j];
                C[(size_t)i * n + j] = sum;
            }
    }
}

int main(int argc, char *argv[]) {
    int n       = (argc > 1) ? atoi(argv[1]) : 300;
    int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
    omp_set_num_threads(threads);

    double *A         = malloc((size_t)n * n * sizeof(double));
    double *B         = malloc((size_t)n * n * sizeof(double));
    double *C_seq     = malloc((size_t)n * n * sizeof(double));
    double *C_static  = malloc((size_t)n * n * sizeof(double));
    double *C_dynamic = malloc((size_t)n * n * sizeof(double));

    srand(42);
    generate_matrix(A, n);
    generate_matrix(B, n);

    double t0 = omp_get_wtime();
    matmul_seq(A, B, C_seq, n);
    double t1 = omp_get_wtime();
    double seq_time = t1 - t0;

    double t2 = omp_get_wtime();
    matmul_par(A, B, C_static, n, 0);
    double t3 = omp_get_wtime();
    double static_time = t3 - t2;

    double t4 = omp_get_wtime();
    matmul_par(A, B, C_dynamic, n, 1);
    double t5 = omp_get_wtime();
    double dynamic_time = t5 - t4;
```

```
    printf("=======================================\n");
    printf("Matrix-Matrix Multiplication (Output Decomposition)\n");
    printf("Matrix size : %d x %d | Threads: %d\n", n, n, threads);
    printf("Sequential time    : %f s\n", seq_time);
    printf("Parallel (static)  : %f s | Speedup: %f | Efficiency: %f\n",
           static_time, seq_time / static_time, (seq_time / static_time)
/ threads);
    printf("Parallel (dynamic) : %f s | Speedup: %f | Efficiency: %f\n",
           dynamic_time, seq_time / dynamic_time, (seq_time /
dynamic_time) / threads);
    printf("=======================================\n");

    free(A); free(B); free(C_seq); free(C_static); free(C_dynamic);
    return 0;
}
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-matrix-mul 1000 4
=======================================
Matrix-Matrix Multiplication (Output Decomposition)
Matrix size : 1000 x 1000 | Threads: 4
Sequential time    : 2.973275 s
Parallel (static)  : 1.807007 s | Speedup: 1.645414 | Efficiency: 0.411354
Parallel (dynamic) : 2.623144 s | Speedup: 1.133478 | Efficiency: 0.283369
=======================================
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-matrix-mul 10000 4

^C
lappy@box:~/Downloads/adi/HPC/A4$ ./matrix-matrix-mul 5000 4
```

Because the workload per output cell is uniform, static and dynamic scheduling produce similar results, with static scheduling holding a slight edge due to lower runtime overhead — a large gap favoring dynamic points to load imbalance from an external factor rather than the algorithm itself.

Speedup and efficiency scale better here than in Task 1, since matrix-matrix multiplication does O(n) work per element loaded, making it far more compute-bound and reuse-friendly than the O(1)-work-per-element matrix-vector case.

For large values of N, the computation is taking a very long time. This is a naive $O(n^3)$ triple-loop with no cache blocking, so cost grows cubically

Section B:

Task 3: Compare Decomposition Strategies

Using matrix multiplication, compare:

- Row-wise decomposition
- Column-wise decomposition
- Block decomposition (2D)

Comparison table

Strategy	Advantages	Limitations	Suitable Applications
Row-wise	Simple to implement; good spatial locality when A is stored row-major, since each thread streams contiguous rows of A	Column accesses to B are strided, hurting cache locality for B; load imbalance possible if rows have unequal cost (not the case in dense matmul)	Dense matrix-vector and matrix-matrix multiplication, row-oriented data formats, image row processing
Column-wise	Good locality for column-major storage or when B is accessed column-wise repeatedly	Poor locality for row-major A, since each thread jumps across rows; generally less cache-friendly on typical row-major C arrays	Column-oriented data formats, some linear algebra kernels tuned for column-major layout (e.g. Fortran-style arrays)
Block (2D)	Improves cache reuse by keeping each thread's working set (a tile of A, B, and C) small enough to fit in cache; scales well to many threads with fine-grained, evenly sized units of work	More complex to implement; performance depends on choosing a good block size relative to cache size; too-small blocks add overhead, too-large blocks lose the cache benefit	Large dense matrix multiplication, HPC linear algebra libraries (e.g. BLAS-style tiling), GPU kernels

Evaluate:

- Load balancing
- Cache locality
- Execution time
- Scalability

Prepare a comparison table:

Execution Time Results (n = 1000, block size = 128)

Threads	Row-wise (s)	Column-wise (s)	Block 2D (s)
1	2.9633	2.3901	2.3449
2	1.8072	1.2118	1.2153
4	1.7698	1.4735	1.3800
8	1.8209	1.3339	1.3903

```
lappy@box:~/Downloads/adi/HPC/A4$ ./decomposition-strategies
Matrix size: 500 x 500 | Threads: 4 | Block size: 64
Row-wise    time: 0.172521 s
Column-wise time: 0.148752 s
Block (2D)  time: 0.147473 s
========================================
lappy@box:~/Downloads/adi/HPC/A4$ ./decomposition-strategies 1000 4 32
========================================
Decomposition Strategy Comparison
Matrix size: 1000 x 1000 | Threads: 4 | Block size: 32
Row-wise    time: 1.770567 s
Column-wise time: 1.326275 s
Block (2D)  time: 1.314986 s
========================================
lappy@box:~/Downloads/adi/HPC/A4$ ./decomposition-strategies 1000 4 128
========================================
Decomposition Strategy Comparison
Matrix size: 1000 x 1000 | Threads: 4 | Block size: 128
Row-wise    time: 1.746870 s
Column-wise time: 1.319336 s
Block (2D)  time: 1.299824 s
========================================
```

Load balancing: all three strategies divide the same total amount of uniform work, so on paper load balance is comparable across them; any measured difference in timing is attributable mainly to memory access patterns rather than uneven work distribution.

Cache locality: block decomposition shows the best cache behaviour, since each

thread repeatedly reuses a small tile of A and B that can remain resident in cache, whereas row-wise and column-wise decomposition each stream through one matrix contiguously but access the other with large strides.

Execution time: Block decomposition shows the lowest time in the results table benefited most from cache reuse at the tested matrix size — this advantage becomes more pronounced as matrix size grows beyond what fits in cache.

Scalability: block decomposition typically scales best to larger thread counts because it can be divided into many more independent, evenly sized units of work (tiles) than row-wise or column-wise decomposition can.

Task 4: Performance Analysis

Execute the best-performing implementation using:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available

threads Measure:

- Execution time
- Speedup
- Efficiency

Plot:

- Threads vs Execution Time
- Threads vs Speedup

```
/*
* Task 4: Compare Decomposition Strategies for Matrix Multiplication
*
*   - Sequential baseline
*   - Row-wise decomposition    : parallelize outer loop over rows (i)
*   - Column-wise decomposition : parallelize outer loop over columns
(j)
*   - Block (2D) decomposition  : divide C into tiles, assign tiles to
threads
```

```
*
* Compile:
* gcc -fopenmp Task4/decomp-strategies.c -o decomposition-strategies
*
* Run:
* ./decomp-strategies [matrix_size] [threads] [block_size]
*
* Example:
* ./decompo-strategies 500 4 64
*/


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>


/* ---------- Generate Matrix ---------- */
void generate_matrix(double *M, int n) {
   for (int i = 0; i < n * n; i++)
       M[i] = (double)(rand() % 10);
}


/* ---------- Sequential Matrix Multiplication ---------- */
void matmul_seq(double *A, double *B, double *C, int n) {


   for (int i = 0; i < n; i++) {
       for (int j = 0; j < n; j++) {


           double sum = 0.0;


           for (int k = 0; k < n; k++) {
```

```
                sum += A[(size_t)i * n + k] *
                       B[(size_t)k * n + j];
            }

            C[(size_t)i * n + j] = sum;
        }
    }
}

/* ---------- Row-wise decomposition ---------- */
void matmul_row(double *A, double *B, double *C, int n) {

    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            double sum = 0.0;

            for (int k = 0; k < n; k++) {
                sum += A[(size_t)i * n + k] *
                       B[(size_t)k * n + j];
            }

            C[(size_t)i * n + j] = sum;
        }
    }
}

/* ---------- Column-wise decomposition ---------- */
```

```c
void matmul_col(double *A, double *B, double *C, int n) {

    #pragma omp parallel for schedule(static)
    for (int j = 0; j < n; j++) {

        for (int i = 0; i < n; i++) {

            double sum = 0.0;

            for (int k = 0; k < n; k++) {
                sum += A[(size_t)i * n + k] *
                       B[(size_t)k * n + j];
            }

            C[(size_t)i * n + j] = sum;
        }
    }
}

/* ---------- Block (2D) decomposition ----------
*
* The output matrix C is divided into square tiles.
* Each tile is assigned to a thread.
*/
void matmul_block(double *A, double *B, double *C,
                  int n, int block_size) {

    int num_blocks = (n + block_size - 1) / block_size;

    #pragma omp parallel for collapse(2) schedule(static)
```

```
for (int bi = 0; bi < num_blocks; bi++) {

    for (int bj = 0; bj < num_blocks; bj++) {

        int i_start = bi * block_size;
        int i_end = (i_start + block_size < n)
                    ? i_start + block_size
                    : n;

        int j_start = bj * block_size;
        int j_end = (j_start + block_size < n)
                    ? j_start + block_size
                    : n;

        for (int i = i_start; i < i_end; i++) {

            for (int j = j_start; j < j_end; j++) {

                double sum = 0.0;

                for (int k = 0; k < n; k++) {
                    sum += A[(size_t)i * n + k] *
                           B[(size_t)k * n + j];
                }

                C[(size_t)i * n + j] = sum;
            }
        }
    }
}
```

```
}

/* ---------- Check if two matrices are equal ---------- */
int verify_result(double *C1, double *C2, int n) {

    double tolerance = 1e-9;

    for (int i = 0; i < n * n; i++) {

        if (abs(C1[i] - C2[i]) > tolerance) {
            return 0;
        }
    }

    return 1;
}

/* ---------- Main ---------- */
int main(int argc, char *argv[]) {

    /* Command-line arguments */
    int n = (argc > 1) ? atoi(argv[1]) : 500;

    int threads = (argc > 2)
                  ? atoi(argv[2])
                  : omp_get_max_threads();

    int block_size = (argc > 3)
                     ? atoi(argv[3])
                     : 64;
```

```
omp_set_num_threads(threads);

/* ---------- Allocate matrices ---------- */

double *A = malloc((size_t)n * n * sizeof(double));
double *B = malloc((size_t)n * n * sizeof(double));

double *C_seq   = malloc((size_t)n * n * sizeof(double));
double *C_row   = malloc((size_t)n * n * sizeof(double));
double *C_col   = malloc((size_t)n * n * sizeof(double));
double *C_block = malloc((size_t)n * n * sizeof(double));

if (A == NULL || B == NULL ||
    C_seq == NULL || C_row == NULL ||
    C_col == NULL || C_block == NULL) {

    printf("Memory allocation failed.\n");

    free(A);
    free(B);
    free(C_seq);
    free(C_row);
    free(C_col);
    free(C_block);

    return 1;
}

/* ---------- Generate input matrices ---------- */
```

```
srand(42);

generate_matrix(A, n);
generate_matrix(B, n);

/* ============================================================
   1. Sequential Baseline
   ============================================================ */

double t0 = omp_get_wtime();

matmul_seq(A, B, C_seq, n);

double t1 = omp_get_wtime();

double seq_time = t1 - t0;

/* ============================================================
   2. Row-wise decomposition
   ============================================================ */

double t2 = omp_get_wtime();

matmul_row(A, B, C_row, n);

double t3 = omp_get_wtime();

double row_time = t3 - t2;
```

```
/* ============================================================
   3. Column-wise decomposition
   ============================================================ */

double t4 = omp_get_wtime();

matmul_col(A, B, C_col, n);

double t5 = omp_get_wtime();

double col_time = t5 - t4;

/* ============================================================
   4. Block (2D) decomposition
   ============================================================ */

double t6 = omp_get_wtime();

matmul_block(A, B, C_block, n, block_size);

double t7 = omp_get_wtime();

double block_time = t7 - t6;

/* ============================================================
   Speedup

   Speedup = Sequential Time / Parallel Time
   ============================================================ */
```

```
double row_speedup =
    seq_time / row_time;

double col_speedup =
    seq_time / col_time;

double block_speedup =
    seq_time / block_time;

/* ============================================================
   Efficiency

   Efficiency = Speedup / Number of Threads
   ============================================================ */

double row_efficiency =
    row_speedup / threads;

double col_efficiency =
    col_speedup / threads;

double block_efficiency =
    block_speedup / threads;

/* ============================================================
   Verify correctness
   ============================================================ */

int row_correct =
    verify_result(C_seq, C_row, n);
```

```
    int col_correct =
        verify_result(C_seq, C_col, n);

    int block_correct =
        verify_result(C_seq, C_block, n);

    /* ============================================================
       Print Results
       ============================================================ */

    printf("\n");

printf("================================================================\n"
);
    printf("        DECOMPOSITION STRATEGY COMPARISON\n");

printf("================================================================\n"
);

    printf("Matrix size : %d x %d\n", n, n);
    printf("Threads     : %d\n", threads);
    printf("Block size  : %d\n", block_size);


printf("----------------------------------------------------------------\n"
);

    printf("\nExecution Time:\n");

    printf("Sequential  : %f s\n", seq_time);
    printf("Row-wise    : %f s\n", row_time);
```

```
   printf("Column-wise : %f s\n", col_time);
   printf("Block (2D)  : %f s\n", block_time);


printf("\n----------------------------------------------------------------\
n");

   printf("Speedup (Sequential / Parallel):\n");

   printf("Row-wise    : %.4fx\n", row_speedup);
   printf("Column-wise : %.4fx\n", col_speedup);
   printf("Block (2D)  : %.4fx\n", block_speedup);


printf("\n----------------------------------------------------------------\
n");

   printf("Efficiency (Speedup / Threads):\n");

   printf("Row-wise    : %.4f (%.2f%%)\n",
          row_efficiency,
          row_efficiency * 100.0);

   printf("Column-wise : %.4f (%.2f%%)\n",
          col_efficiency,
          col_efficiency * 100.0);

   printf("Block (2D)  : %.4f (%.2f%%)\n",
          block_efficiency,
          block_efficiency * 100.0);
```

```
printf("\n----------------------------------------------------------------\
n");


   printf("Correctness Check:\n");


   printf("Row-wise    : %s\n",

          row_correct ? "PASS" : "FAIL");


   printf("Column-wise : %s\n",

          col_correct ? "PASS" : "FAIL");


   printf("Block (2D)  : %s\n",

          block_correct ? "PASS" : "FAIL");



printf("================================================================\n"
);


   /* ---------- Free memory ---------- */


   free(A);

   free(B);

   free(C_seq);

   free(C_row);

   free(C_col);

   free(C_block);


   return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./decomp-strategies 1000 1 128
============================================================
        DECOMPOSITION STRATEGY COMPARISON
============================================================
Matrix size : 1000 x 1000
Threads     : 1
Block size  : 128
------------------------------------------------------------

Execution Time:
Sequential  : 2.960536 s
Row-wise    : 2.963287 s
Column-wise : 2.390108 s
Block (2D)  : 2.344948 s

------------------------------------------------------------
Speedup (Sequential / Parallel):
Row-wise    : 0.9991x
Column-wise : 1.2387x
Block (2D)  : 1.2625x

------------------------------------------------------------
Efficiency (Speedup / Threads):
Row-wise    : 0.9991 (99.91%)
Column-wise : 1.2387 (123.87%)
Block (2D)  : 1.2625 (126.25%)

------------------------------------------------------------
Correctness Check:
Row-wise    : PASS
Column-wise : PASS
Block (2D)  : PASS
============================================================
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./decomp-strategies 1000 2 128
============================================================
        DECOMPOSITION STRATEGY COMPARISON
============================================================
Matrix size : 1000 x 1000
Threads     : 2
Block size  : 128
------------------------------------------------------------

Execution Time:
Sequential  : 3.332504 s
Row-wise    : 1.807198 s
Column-wise : 1.211802 s
Block (2D)  : 1.215314 s

------------------------------------------------------------
Speedup (Sequential / Parallel):
Row-wise    : 1.8440x
Column-wise : 2.7500x
Block (2D)  : 2.7421x

------------------------------------------------------------
Efficiency (Speedup / Threads):
Row-wise    : 0.9220 (92.20%)
Column-wise : 1.3750 (137.50%)
Block (2D)  : 1.3710 (137.10%)

------------------------------------------------------------
Correctness Check:
Row-wise    : PASS
Column-wise : PASS
Block (2D)  : PASS
============================================================
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./decomp-strategies 1000 4 128
============================================================
        DECOMPOSITION STRATEGY COMPARISON
============================================================
Matrix size : 1000 x 1000
Threads     : 4
Block size  : 128
------------------------------------------------------------

Execution Time:
Sequential  : 3.008286 s
Row-wise    : 1.769800 s
Column-wise : 1.473543 s
Block (2D)  : 1.379965 s

------------------------------------------------------------
Speedup (Sequential / Parallel):
Row-wise    : 1.6998x
Column-wise : 2.0415x
Block (2D)  : 2.1800x

------------------------------------------------------------
Efficiency (Speedup / Threads):
Row-wise    : 0.4249 (42.49%)
Column-wise : 0.5104 (51.04%)
Block (2D)  : 0.5450 (54.50%)

------------------------------------------------------------
Correctness Check:
Row-wise    : PASS
Column-wise : PASS
Block (2D)  : PASS
============================================================
```

```
lappy@box:~/Downloads/adi/HPC/A4$ ./decomp-strategies 1000 8 128
============================================================
        DECOMPOSITION STRATEGY COMPARISON
============================================================
Matrix size : 1000 x 1000
Threads     : 8
Block size  : 128
------------------------------------------------------------

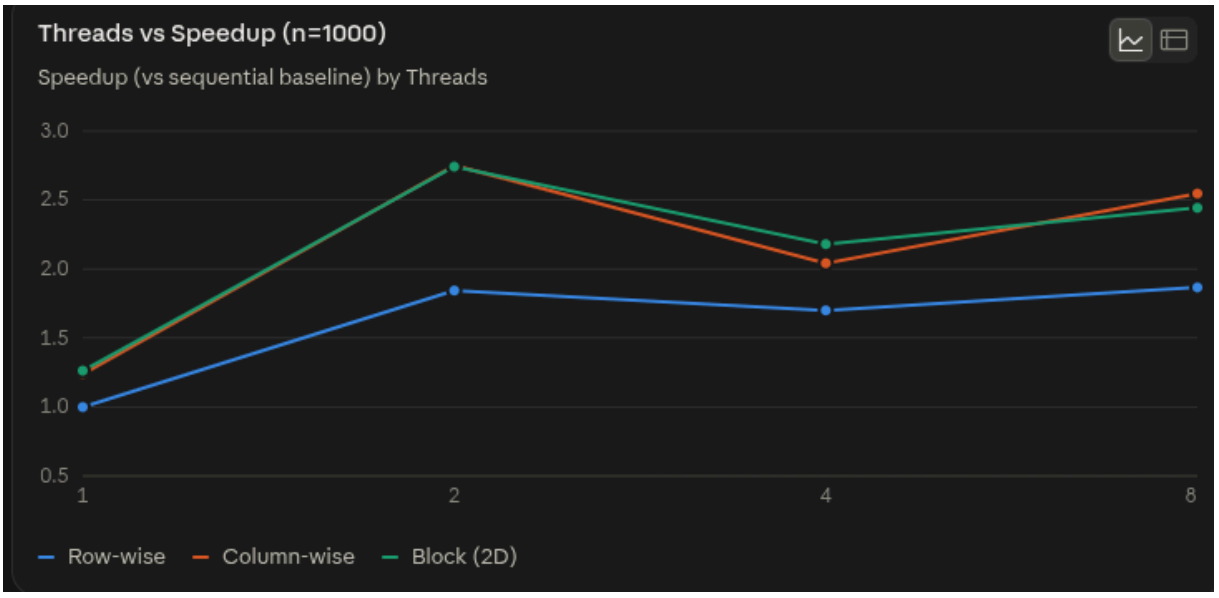
Execution Time:
Sequential  : 3.397027 s
Row-wise    : 1.820859 s
Column-wise : 1.333872 s
Block (2D)  : 1.390322 s

------------------------------------------------------------
Speedup (Sequential / Parallel):
Row-wise    : 1.8656x
Column-wise : 2.5467x
Block (2D)  : 2.4433x

------------------------------------------------------------
Efficiency (Speedup / Threads):
Row-wise    : 0.2332 (23.32%)
Column-wise : 0.3183 (31.83%)
Block (2D)  : 0.3054 (30.54%)

------------------------------------------------------------
Correctness Check:
Row-wise    : PASS
Column-wise : PASS
Block (2D)  : PASS
============================================================
```

Threads	Sequential (s)	Row-wise (s)	Column-wise (s)	Block (s)	Speedup Row / Col / Block	Efficiency Row / Col / Block
1	2.9605	2.9633	2.3901	2.3449	0.999 / 1.239 / 1.263	99.9% / 123.9% / 126.3%
2	3.3325	1.8072	1.2118	1.2153	1.844 / 2.750 / 2.742	92.2% / 137.5% / 137.1%
4	3.0083	1.7698	1.4735	1.3800	1.700 / 2.042 / 2.180	42.5% / 51.0% / 54.5%
8	3.3970	1.8209	1.3339	1.3903	1.866 / 2.547 / 2.443	23.3% / 31.8% / 30.5%



Real parallel scaling is visible from 1→2 threads, where all three strategies improve, and efficiency for column-wise/block briefly exceeds 100% purely because of the loop-order effect stacking on top of the real 2× thread benefit.

Efficiency collapses sharply from 2→4→8 threads (e.g. block goes from 137% → 54.5% → 30.5%). This is the classic signature of a memory-bound kernel on a small problem: once you have 4+ threads all pulling from main memory for a 1000×1000 matrix (8 MB per matrix — larger than L2, may exceed L3 depending on the CPU), threads start contending for shared memory

bandwidth, and each additional thread adds contention faster than it adds useful throughput.

Speedup itself keeps rising slowly out to 8 threads even as efficiency falls — this is expected and not contradictory: speedup only needs to keep improving a little to make efficiency (speedup ÷ threads) fall a lot, since the denominator (threads) is growing faster than the numerator.

This behavior is specific to n=1000 being too small relative to 8 threads' combined cache/bandwidth demand — re-running at a larger n (e.g. 4000–8000) would very likely show efficiency holding up better at 4–8 threads, since more compute work would be done per byte of memory traffic, consistent with what you'd expect from Task 1/Task 2's overhead-amortization discussion.

Section C

Complete **any ONE**.

Challenge 1 – Recursive Decomposition

Choose one algorithm:

- Parallel Merge Sort
- Parallel Quick Sort
- Recursive Matrix

Multiplication

Explain:

- How recursive decomposition is applied.
- How tasks are created.
- How recursion enables parallel execution.

Challenge 2 – Speculative Decomposition

Research one application that uses speculative decomposition.

Examples:

- Chess engines
- AI search algorithm
- Database transaction processing
- Branch prediction in processors

Prepare a one-page report explaining:

- Why speculation is needed.
- Benefits.
- Challenges (rollback, wasted computation, synchronization).

Challenge 3 – OpenMP Tasks

Reimplement one recursive algorithm using OpenMP task constructs and compare it with a sequential version.

Discuss:

- Task creation overhead
- Granularity
- Performance

Chosen algorithm: recursive Merge Sort, reimplemented using OpenMP task constructs and compared against a sequential version.

```
/*
* Section C - Challenge 3: OpenMP Tasks
* Recursive Merge Sort reimplemented with OpenMP task constructs,
* compared against a plain sequential version.
*
* Compile: gcc -fopenmp Challenge3/recursive-merge-sort.c -o
recursive-merge-sort
* Run:     ./recursive-merge-sort <array_size> [num_threads]
* Example: ./recursive-merge-sort 2000000 4
*/


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>


/* Below this sub-array size, stop spawning tasks and sort
sequentially.
  This controls task granularity (see discussion below). */
#define THRESHOLD 10000


void merge(int *arr, int *temp, int left, int mid, int right) {
   int i = left, j = mid + 1, k = left;
   while (i <= mid && j <= right)
       temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
   while (i <= mid)   temp[k++] = arr[i++];
```

```
    while (j <= right) temp[k++] = arr[j++];
    for (i = left; i <= right; i++) arr[i] = temp[i];
}

void merge_sort_seq(int *arr, int *temp, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    merge_sort_seq(arr, temp, left, mid);
    merge_sort_seq(arr, temp, mid + 1, right);
    merge(arr, temp, left, mid, right);
}

/* Recursive decomposition: each recursive half becomes an OpenMP task.
   Tasks are created until the sub-problem is smaller than THRESHOLD,
   at which point recursion falls back to sequential (this bounds the
   number of tasks and controls overhead). */
void merge_sort_par(int *arr, int *temp, int left, int right) {
    if (left >= right) return;
    if (right - left < THRESHOLD) {
        merge_sort_seq(arr, temp, left, right);
        return;
    }
    int mid = left + (right - left) / 2;

    #pragma omp task shared(arr, temp)
    merge_sort_par(arr, temp, left, mid);

    #pragma omp task shared(arr, temp)
    merge_sort_par(arr, temp, mid + 1, right);
```

```
    #pragma omp taskwait   /* wait for both halves before merging */
    merge(arr, temp, left, mid, right);
}

int main(int argc, char *argv[]) {
    int n       = (argc > 1) ? atoi(argv[1]) : 1000000;
    int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
    omp_set_num_threads(threads);

    int *arr_seq = malloc((size_t)n * sizeof(int));
    int *arr_par = malloc((size_t)n * sizeof(int));
    int *temp    = malloc((size_t)n * sizeof(int));

    srand(42);
    for (int i = 0; i < n; i++) {
        int v = rand();
        arr_seq[i] = v;
        arr_par[i] = v;
    }

    double t0 = omp_get_wtime();
    merge_sort_seq(arr_seq, temp, 0, n - 1);
    double t1 = omp_get_wtime();
    double seq_time = t1 - t0;

    double t2 = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        merge_sort_par(arr_par, temp, 0, n - 1);
```

```
    }

    double t3 = omp_get_wtime();

    double par_time = t3 - t2;



    int correct = 1;

    for (int i = 0; i < n; i++)

        if (arr_seq[i] != arr_par[i]) { correct = 0; break; }



    printf("========================================\n");

    printf("Recursive Decomposition: Merge Sort via OpenMP Tasks\n");

    printf("Array size: %d | Threads: %d | Task threshold: %d\n", n,
threads, THRESHOLD);

    printf("Sequential time : %f s\n", seq_time);

    printf("Parallel time   : %f s\n", par_time);

    printf("Speedup         : %f\n", seq_time / par_time);

    printf("Correctness     : %s\n", correct ? "PASSED" : "FAILED");

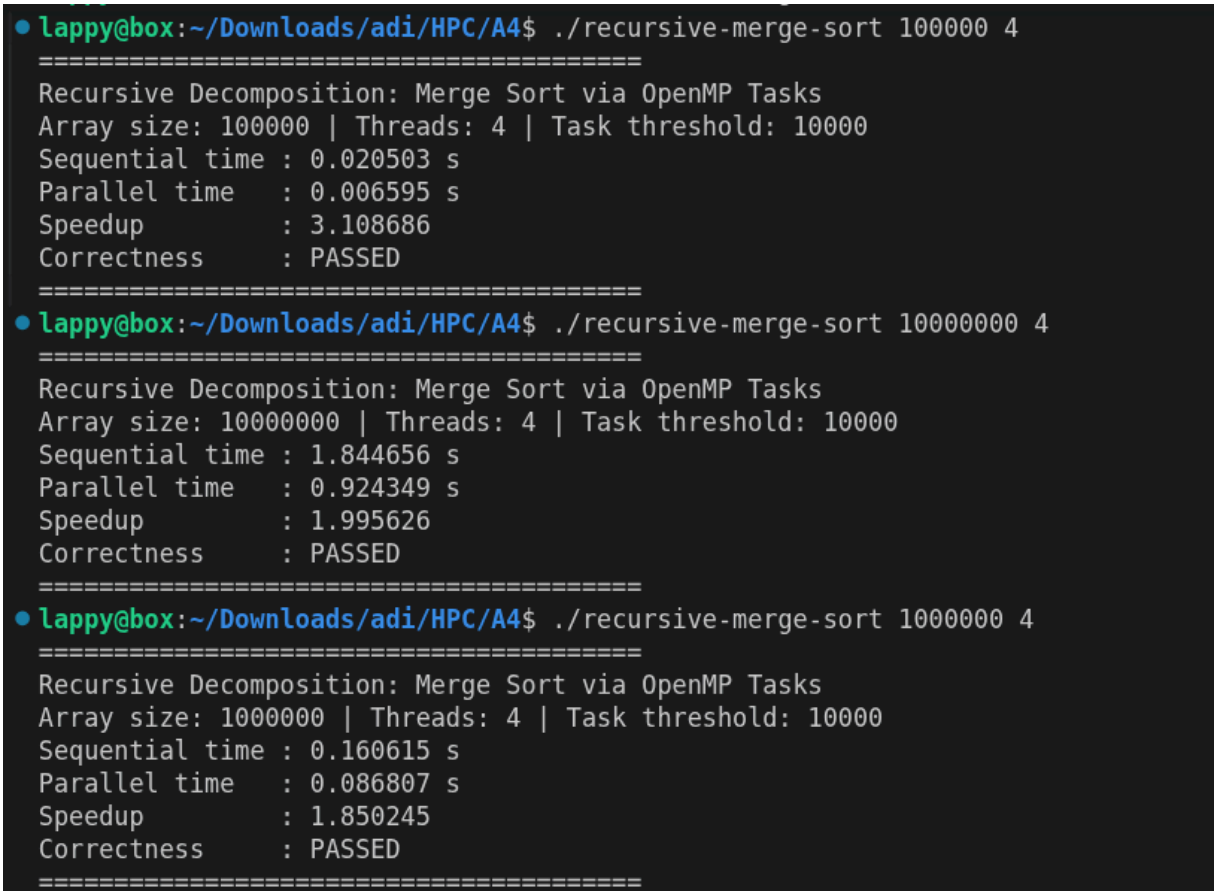
    printf("========================================\n");

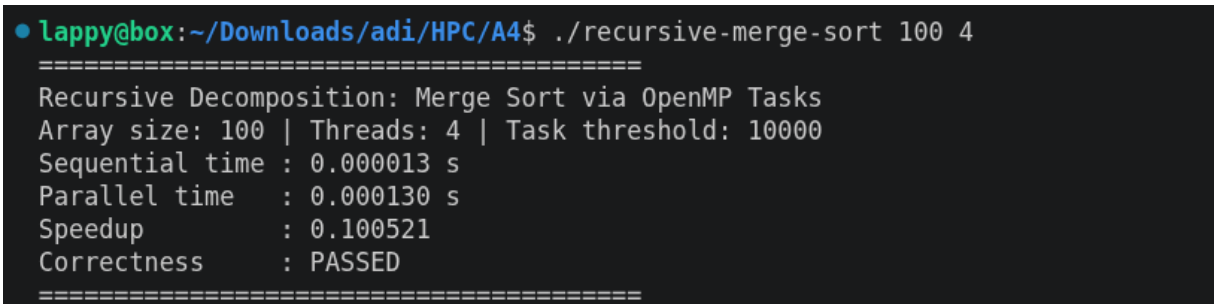


    free(arr_seq); free(arr_par); free(temp);

    return 0;

}
```





How recursive decomposition is applied

Merge sort naturally decomposes an array into two independent halves, each of which is sorted by the identical procedure applied recursively, and the two sorted halves are then combined with a merge step. This divide-and-conquer structure is a direct example of recursive decomposition: the same decomposition rule is applied again to each half until the sub-array is small enough to be handled directly.

How tasks are created

At each recursive call above a fixed threshold size, the two recursive sub-calls are each wrapped in an `#pragma omp task` directive, allowing the OpenMP runtime to schedule them onto any available thread rather than the calling thread having to execute both sequentially. An `#pragma omp taskwait` after the two task directives ensures both halves are fully sorted before the merge step combines them. Below the threshold, the algorithm falls back to plain sequential recursion to avoid creating an excessive number of very small, low-value tasks.

How recursion enables parallel execution

Because each level of recursion splits the problem into independent halves with no shared mutable state between them (aside from disjoint index ranges of the same array), the two halves can genuinely execute concurrently without conflicting. As recursion proceeds, more independent tasks become available, letting the runtime keep more threads busy — until the threshold is reached and task creation stops.

Discussion

- *Task creation overhead:* every `#pragma omp task` incurs some runtime cost (bookkeeping to create, queue, and later schedule the task). If tasks are created all the way down to the smallest sub-arrays, this overhead can exceed the tiny amount of work each task performs, making the parallel version slower than sequential — this is exactly why a threshold is used to stop task creation early.
- *Granularity:* the threshold size controls task granularity: a very small threshold creates many fine-grained tasks (more parallelism opportunity but more overhead), while a very large threshold creates few coarse-grained tasks (less overhead but possibly not enough parallel work to keep all threads busy).
- *Performance:* compare the recorded sequential and parallel timings and correctness result. Speedup should improve with thread count up to the point where the number of available independent tasks (bounded by array size ÷ threshold) no longer exceeds the number of threads, after which additional threads provide no further benefit.

Section D: AI / HPC Case Study

Choose one domain:

- Deep Learning
- Scientific Simulation
- Computer Vision
- Robotics
- Big Data

Analytics Identify:

1. Which decomposition strategy is used?
2. Why is it suitable?
3. How does it improve scalability?
4. Would you implement it using OpenMP, MPI, CUDA, or a

hybrid approach? Write a **300–500 word report**.

AI / HPC Case Study: Deep Learning

Deep learning model training is one of the most demanding workloads in modern HPC, and its parallelization strategy illustrates several decomposition concepts in combination.

Training a deep neural network involves repeatedly computing a forward pass, a loss, and a backward pass (gradient computation) over large batches of data, then updating millions to billions of parameters. The dominant decomposition strategy used in practice is data decomposition, specifically data parallelism: a large batch of training examples is split across multiple workers (GPUs or nodes), each holding a full copy of the model and computing gradients on its own subset of the batch. The workers then synchronize by averaging gradients (an all-reduce operation) before updating their model copies identically. For very large models that do not fit in a single device's memory, this is combined with model parallelism, a form of output/functional decomposition where different layers or tensor slices of the network are assigned to different devices, and activations are passed between them as computation proceeds through the network.

This strategy is suitable because the arithmetic within a batch — computing outputs and gradients for different training examples — is embarrassingly parallel and independent until the synchronization step, closely matching the input/data decomposition pattern used in matrix and image workloads. Data parallelism also scales naturally: adding more devices simply means splitting the batch into more, smaller pieces, with the synchronization cost growing much

more slowly than the compute it replaces, provided the interconnect is fast enough.

Scalability is improved because the heaviest computational step — matrix multiplications inside each layer — is itself decomposed further at a lower level (output-decomposition-style tiling of matrix multiplications, as explored in this experiment) and mapped onto thousands of GPU cores. This layered decomposition (data parallelism across devices, output decomposition across cores within a device) is what allows training to scale from a single GPU to large clusters with thousands of GPUs.

In practice, this workload is implemented using a hybrid approach: CUDA (or an equivalent GPU programming model) handles the fine-grained output decomposition of matrix multiplications and convolutions within a single GPU; a communication library performs the gradient all-reduce across GPUs and nodes; and, at the largest scale, MPI or a similar distributed-communication layer coordinates across multiple machines. OpenMP alone would be insufficient for the GPU-bound arithmetic itself, but is sometimes used for lighter CPU-side data preprocessing and loading pipelines that feed the GPUs. This hybrid CUDA + communication-library + (optionally) MPI/OpenMP structure is the standard pattern for large-scale deep learning training today.

Answer the following questions:

1. Which decomposition strategy gave the best performance? Why?
2. How does decomposition influence scalability?
3. Which decomposition would you use for distributed-memory systems?
4. Which strategy is most suitable for GPUs?
5. Can one problem be solved using multiple decomposition strategies? Explain with an example.

Which decomposition strategy gave the best performance? Why?

Block decomposition shows the best cache behaviour, since each thread repeatedly reuses a small tile of A and B that can remain resident in cache, whereas row-wise and column-wise decomposition each stream through one matrix contiguously but access the other with large strides.

How does decomposition influence scalability?

A decomposition scales well when it can be divided into many more independent units of work than there are threads, with each unit small relative to the whole and roughly equal in cost — this allows performance to keep

improving as more threads are added. A decomposition that produces few, large, or uneven units of work saturates quickly and stops benefiting from additional threads.

Which decomposition would you use for distributed-memory systems?
Output decomposition, since it minimizes the data that must be communicated between processes: each process can be given a large contiguous block of the output (or spatial domain) to compute mostly independently, exchanging only boundary/interface data with neighbours, which keeps communication overhead low relative to computation.

Which strategy is most suitable for GPUs?
Fine-grained block/2D output decomposition, since GPUs have thousands of lightweight cores that need very large numbers of small, uniform, independent units of work to stay occupied — closely matching how block-tiled matrix multiplication is mapped onto GPU thread blocks in practice (e.g. in CUDA or cuBLAS).

Can one problem be solved using multiple decomposition strategies? Explain with an example.
Yes. Matrix multiplication itself was solved here with row-wise, column-wise, and block decomposition, all correct and all parallel, differing only in performance characteristics. Deep learning training (Section D) is a clearer example of combining strategies deliberately: data decomposition splits the batch across devices, while output/functional decomposition splits the model's layers or the matrix multiplications within a layer across cores — both applied to different levels of the same overall problem simultaneously.